


# COS30082 Applied Machine Learning
# Assignment - Bird Species Classification
# Due: 11:59 pm on 23/10/2026

Name: Andy Wei Jian CHU
Student ID: 104401283
GitHub: https://github.com/AA-4499/Bird-Species-Classification.git
Stream lit: https://bird-species-classification-nm4sapwjfrwxp2gzro6cin.streamlit.app/
YouTube: https://youtu.be/-bcM3VG2heM?si=Lo_1ywcedldrziFq

## Introduction

This report tackles fine-grained, 200-class bird species classification on the Caltech-UCSD Birds-200 (CUB-200) dataset, which provides 4,829 training images and 1,204 official test images. With only about 24 training images per class and a very high-dimensional input space, a network trained from scratch would have low bias but high variance: it would tend to memorise the training set and generalise poorly, so overfitting is the core machine-learning risk. We use deep transfer learning through inductive transfer: feature representations learned on a source domain and task (DS, TS), ImageNet image classification, are reused for the target domain and task (DT, TT), 200-way bird recognition, where labelled target data are available. A ResNet-50 baseline and a ConvNeXt-Tiny model are each trained as a frozen feature extractor and as a fine-tuned network (three seeds each), together with data augmentation, label smoothing, dropout and weight decay as regularises. Fine-tuned ConvNeXt-Tiny performs best, reaching 89.28 ± 0.45% Top 1 accuracy on the test set.

## Section 1 – Methodology

### 1.1 Dataset and data split

In the Bird-200 (CUB-200) dataset the task is single-label, 200-class fine-grained classification; there are 4,829 training image (about 24 per species) and 1,204 official test images (around 6 per species). Image varies in pose, scale, viewpoint, lighting, background and occlusion, and many species differ only in small cues such as bill shape, eye-rings or wing bars. With so few images

per class, overfitting is the main risk. The 4,829 training images are split with a stratified 85/15 split (random_state=42) into 4,104 training and 725 validation images, so every species is represented in both parts. The validation set is used for checkpoint selection and model comparison; the official test set is used only for final evaluation.

1.2 Preprocessing and augmentation

All images are converted to RGB and fed to the networks at 224×224. Training pipeline: Resize(256×256) →RandomResizedCrop(224, scale 0.5 – 1.0) → RandomHorizontalFlip(p=0.5) → ColorJitter(brightness 0.2, contrast 0.2, saturation 0.2, hue 0.05) → ToTensor → ImageNet normalisation (mean 0.485/0.456/0.406, std 0.229/0.224/0.225) → RandomErasing(p=0.2, area 2 – 20%). Validation/test pipeline is deterministic: Resize(256×256) → CenterCrop(224) → ToTensor → ImageNet normalisation so, evaluation is reproducible. The augmentations target specific failure sources: cropping varies scale and position, so the model does not rely on a fixed composition; flipping adds pose variation (species identity does not depend on facing direction); colour jitter reduces sensitivity to lighting; random erasing simulates occlusion by leaves and branches and discourages dependence on one cue. ImageNet normalisation matches the statistics the pretrained backbones expect. Beyond augmentation, the other regularises are label smoothing, dropout (0.3) in the classifier, AdamW weight decay and a cosine learning-rate decay (Section 1.4).

1.3 Architectures and transfer learning

Baseline – ResNet-50 (ImageNet-1K pretrained): a 50-layer residual network built from bottleneck blocks with skip connections, producing a 2048-d pooled feature. Its 1,000-class layer is replaced by Dropout (0.3) →Linear (2048, 200). Advanced – ConvNeXt-Tiny (timm convnext_tiny. fb_in22k_ft_in1k, i.e. pretrained on ImageNet-22K then fine-tuned on ImageNet-1K): a modernised convolutional network using 7×7 depthwise convolutions, inverted-bottleneck blocks, GELU activations, LayerNorm and a hierarchical design. Its head is re-created for 200 classes with drop_rate=0.3. The code supports two transfer modes: frozen feature extractor (all backbone parameters fixed, only the new head trained) and fine-tuning (backbone and head trained, with a smaller learning rate on the pretrained backbone and a 5× larger one on the new head). Both modes were run for both architectures (Section 2). In frozen mode only the new head is trained, at the head learning rate of 5e-4 (0.0005).

1.4 Loss function, optimisation and hyperparameters

The loss is categorical cross-entropy with label smoothing, nn.CrossEntropyLoss(label_smoothing=0.1), applied to the raw logits (PyTorch combines log-softmax and negative log-likelihood internally; softmax is applied explicitly only at evaluation to obtain confidences). Smoothing (ε = 0.1) stops the model being pushed to probability 1.0 for a single class, which helps with visually similar species. Optimisation uses AdamW (weight decay 0.01) with CosineAnnealingLR decaying every parameter group to 1e-6 ((0.000001)) over the 15 epochs. The best checkpoint is the epoch with the highest validation average class accuracy, never the test accuracy.

| Hyperparameter | Value | Hyperparameter | Value |
|---|---|---|---|
| Epochs / batch size | 15 / 16 | Backbone LR / head LR | 1e-4 / 5e-4 (×5) \| (0.0001 / 0.0005) |
| Optimiser / weight decay | AdamW / 0.01 | Scheduler / min LR | Cosine annealing / 1e-6 (0.000001) |

| | | | |
|---|---|---|---|
| Label smoothing / dropout | 0.1 / 0.3 | Input size / classes | 224×224 / 200 |
| Validation ratio | 0.15 (stratified) | Seeds / precision | 42, 43, 44 / FP16 AMP on CUDA |
| Modes compared | Frozen, fine-tuned | Total runs | 12 (2 models × 2 modes × 3 seeds) |

# Section 2 – Results and Discussion

## 2.1 Performance Comparison

Both metrics required by the brief are reported on the 1,204 official test images: Top-1 accuracy and average accuracy per class (unweighted mean over the 200 per-class accuracies). Each of the four configurations (two architectures × frozen/fine-tuned) was trained three times (seeds 42, 43, 44) with identical hyperparameters apart from which parameters are trainable.

| Model | Mode | Top-1 accuracy (%) | Average class accuracy (%) | Per-seed Top-1 (42/43/44) | Val.avg-class (%) |
|---|---|---|---|---|---|
| ResNet-50 | Frozen | 51.22 ± 0.72 | 50.33 ± 0.80 | 50.75/50.66/52.24 | 52.72 |
| ResNet-50 | Fine-tuned | 80.73 ± 0.24 | 80.16 ± 0.22 | 80.40/80.81/80.98 | 81.81 |
| ConvNeXt-Tiny | Frozen | 86.54 ± 0.13 | 86.39 ± 0.20 | 86.71 / 86.54 / 86.38 | 86.41 |
| ConvNeXt-Tiny | Fine-tuned | 89.28 ± 0.45 | 89.14 ± 0.28 | 89.78 / 89.37 / 88.70 | 88.97 |

## 2.2 Training and validation learning curves

| Model | Final train Top-1 (%) | Final val. Top-1 (%) | Train–val gap (pp) |
|---|---|---|---|
| ResNet-50 | 98.81 | 81.98 | 16.84 |
| ConvNeXt-Tiny | 99.55 | 89.24 | 10.30 |

Both models fit the training set almost perfectly (≈99%), and loss falls smoothly under the cosine schedule. Validation accuracy rises more slowly and fluctuates slightly in the middle epochs, then stabilises as the learning rate decays. ConvNeXt-Tiny starts far ahead (about 74% validation Top-1 after epoch 1 against about 48% for ResNet-50) and converges faster. The train–validation gap is still large for both models (10–17 pp), so the regularisers reduce but do not remove overfitting at roughly 24 images per class; the gap is smaller for ConvNeXt-Tiny. The frozen runs are summarised by their final test and validation scores in Section 2.1 rather than by curves. These curves document the observed behaviour; no ablation was run, so they do not show how much each individual regulariser (augmentation, label smoothing, dropout, weight decay) contributed.

## 2.3 Discussion and model selection

The selected model is a fine-tuned ConvNeXt-Tiny, chosen based on the highest validation average class accuracy rather than test accuracy. Across all 12 runs, the best configuration was ConvNeXt-Tiny fine-tuned with seed 42, achieving 90.53% validation average class accuracy and scoring 89.78% Top-1 and 89.47% average class accuracy on the official test set. Over three seeds, this configuration reaches 89.28 ± 0.45% Top-1 and 89.14 ± 0.28% average class accuracy, outperforming fine-tuned ResNet-50 (80.73 ± 0.24% / 80.16 ± 0.22%) on every seed; the gap is far larger than the seed-to-seed variation (SD below 0.5 pp), and validation scores rank the configurations in the same order. The web application currently serves the earlier ConvNeXt-Tiny

production checkpoint (89.53% / 89.36%), which is consistent with these results. Fine-tuning proved better than frozen features for both architectures, but by very different margins: ResNet-50 improves by about 30 pp (51.2% → 80.7% Top-1), indicating its ImageNet-1K features are a poor fit for 200 fine-grained bird species until the backbone adapts to bird-specific cues such as feather pattern, bill shape, and wing markings. ConvNeXt-Tiny improves by only about 3 pp (86.5% → 89.3%), as its frozen features already separate the species well—a frozen ConvNeXt-Tiny (86.5%) even beats a fine-tuned ResNet-50 (80.7%). Fine-tuning also uses a low backbone learning rate (1e-4) to avoid destroying the pretrained features. Plausible architectural reasons for ConvNeXt-Tiny's strength include its larger 7×7 depthwise receptive field (useful for relating bill, head pattern, and body colour), inverted-bottleneck blocks, and LayerNorm/GELU design. However, the large gap between the frozen models suggests that pretraining matters at least as much: ConvNeXt-Tiny is initialized from ImageNet-22K pretraining while ResNet-50 uses ImageNet-1K only, so architecture and pretraining data are confounded, and this experiment cannot separate them. Additionally, frozen and fine-tuned runs share one hyperparameter setting (15 epochs, head learning rate 5e-4) that was not tuned for the frozen case, so the size of the frozen-versus-fine-tuned gap for ResNet-50 may be somewhat sensitive to that setting.

2.4 Failure-mode and error analysis

Error analysis uses the deployed ConvNeXt-Tiny checkpoint on the official test set (89.53% Top-1). The most frequent confusions are between visually near-identical species: Common Tern → Arctic Tern (3 errors); Chuck-will's-widow ↔ Whip-poor-will (2 each way); Acadian Flycatcher ↔ Least Flycatcher (2 each way); California Gull ↔ Ring-billed Gull (2 each way); plus Brewer's Sparrow → Vesper Sparrow and Warbling Vireo → Philadelphia Vireo (2 each). The weakest classes are Chuck-will's-widow (25.0%) and Common Tern (28.6%), followed by Red-faced Cormorant, Whip-poor-will and Chipping Sparrow (50.0% each). The ResNet-50 baseline shows the same pattern (e.g. Least Flycatcher → Acadian Flycatcher, Whip-poor-will → Chuck-will's-widow, Ring-billed Gull → Herring Gull, 3 errors each) with weaker classes such as Henslow's Sparrow (14.3%) and California Gull (20.0%), so several difficult pairs recur across both models, which suggests they reflect the data rather than one architecture. The mutual (A ↔ B) confusions point to inter-class similarity – near-identical plumage within a genus – as the main failure mode, compounded by intra-class variance (age, sex, pose, distance, lighting). Background clutter and occlusion are plausible additional causes, and random cropping/erasing only partly mitigate them. With only about six test images per class, one or two errors move a class accuracy by 15–25 pp, and no image-level annotations (occlusion, pose, background) were recorded, so these causes are hypotheses; confirming them would need a manual review of the misclassified images.

2.5 Conclusion

Fine-tuned ConvNeXt-Tiny, selected on validation data, gave the best CUB-200 results (89.28 ± 0.45% Top-1 and 89.14 ± 0.28% average class accuracy over three seeds; 89.78% / 89.47% for the validation-selected seed-42 run). It clearly outperforms fine-tuned ResNet-50 (80.73 ± 0.24% / 80.16 ± 0.22%), and fine-tuning beat frozen features for both architectures, by about 30 pp for ResNet-50 and about 3 pp for ConvNeXt-Tiny. Remaining limitations are
the residual train–validation gap, the unequal pretraining of the two backbones, a single untuned hyperparameter setting for the frozen mode and the lack of per-regulariser ablations; these are the natural next experiments.